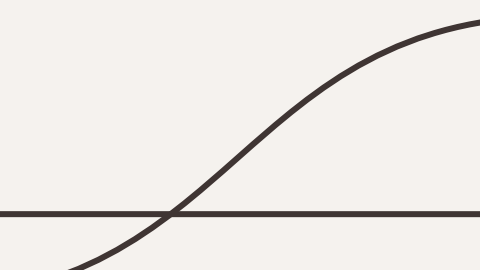


Programación dinámica

Diseño de algoritmos
Jacqueline Aldridge Águila



Fibonacci recursivo

Fibonacci Recursivo

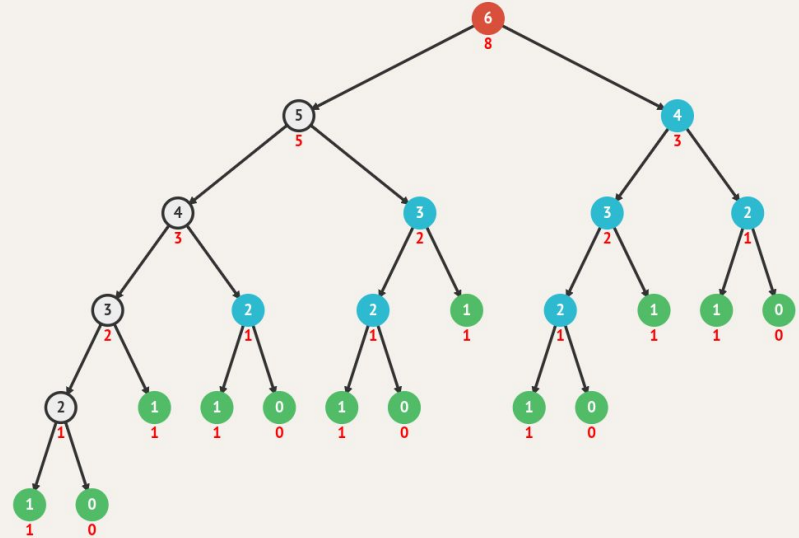
```

1: function RECFIB( $n$ )
2:   if  $n = 0$  then
3:     return 0
4:   else if  $n = 1$  then
5:     return 1
6:   else
7:     return RECFIB( $n - 1$ ) + RECFIB( $n - 2$ )

```

$$T(n) = T(n-1) + T(n-2)$$

$$T(n) \in O(2^n)$$



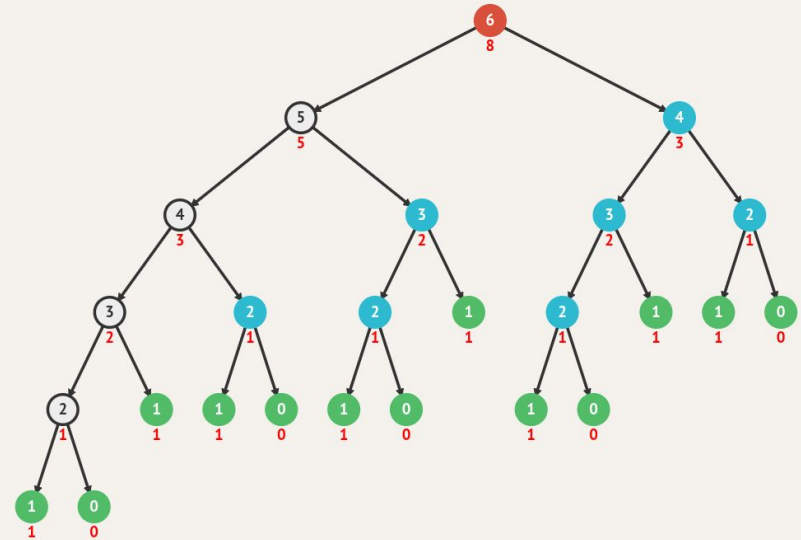
Fibonacci recursivo

Fibonacci Recursivo

```
1: function RECFIB( $n$ )  
2:   if  $n = 0$  then  
3:     return 0  
4:   else if  $n = 1$  then  
5:     return 1  
6:   else  
7:     return RECFIB( $n - 1$ ) + RECFIB( $n - 2$ )
```

$$T(n) = T(n-1) + T(n-2)$$

$$T(n) \in O(2^n)$$



fib(1) se calculó 8 veces

fib(2) se calculó 5 veces

fib(3) se calculó 3 veces

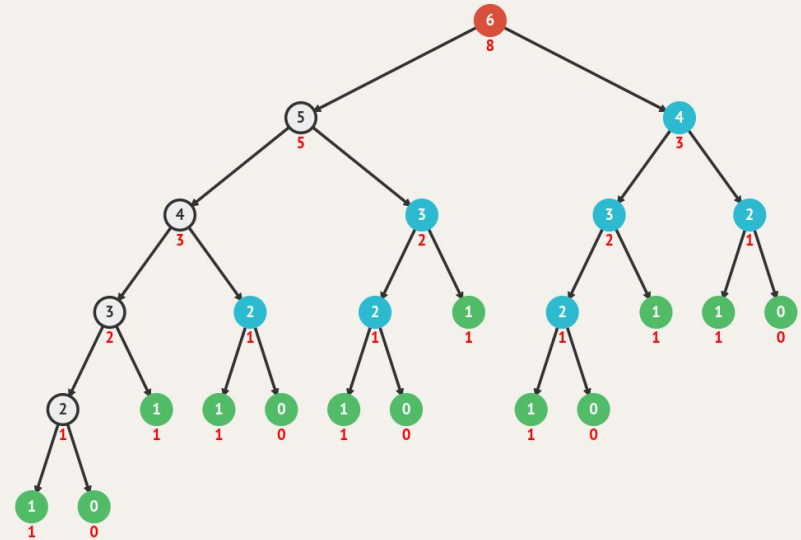
Fibonacci recursivo

Fibonacci Recursivo

```
1: function RECFIB( $n$ )  
2:   if  $n = 0$  then  
3:     return 0  
4:   else if  $n = 1$  then  
5:     return 1  
6:   else  
7:     return RECFIB( $n - 1$ ) + RECFIB( $n - 2$ )
```

$$T(n) = T(n-1) + T(n-2)$$

$$T(n) \in O(2^n)$$



fib(1) se calculó 8 veces

n=20

fib(2) se calculó 5 veces

fib(3) se calculó 3 veces

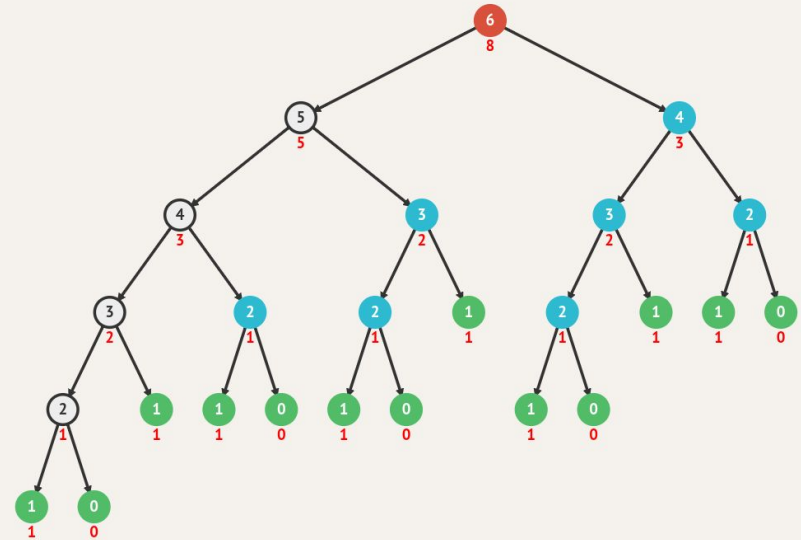
Fibonacci recursivo

Fibonacci Recursivo

```
1: function RECFIB( $n$ )  
2:   if  $n = 0$  then  
3:     return 0  
4:   else if  $n = 1$  then  
5:     return 1  
6:   else  
7:     return RECFIB( $n - 1$ ) + RECFIB( $n - 2$ )
```

$$T(n) = T(n-1) + T(n-2)$$

$$T(n) \in O(2^n)$$



fib(1) se calculó 8 veces

fib(2) se calculó 5 veces

fib(3) se calculó 3 veces

$n=20$ ~22.000 llamadas

Fibonacci recursivo

Fibonacci Recursivo

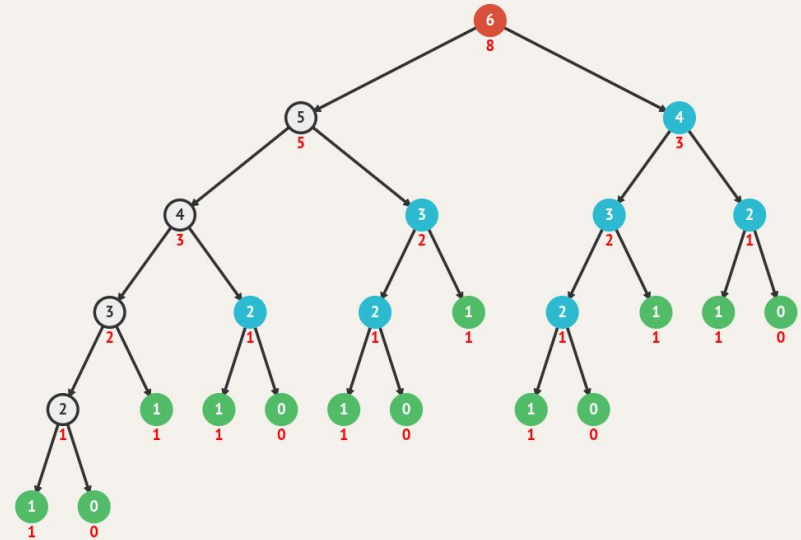
```

1: function RECFIB( $n$ )
2:   if  $n = 0$  then
3:     return 0
4:   else if  $n = 1$  then
5:     return 1
6:   else
7:     return RECFIB( $n - 1$ ) + RECFIB( $n - 2$ )

```

$$T(n) = T(n-1) + T(n-2)$$

$$T(n) \in O(2^n)$$



fib(1) se calculó 8 veces

fib(2) se calculó 5 veces

fib(3) se calculó 3 veces

n=20 ~22.000 llamadas

$n=30$

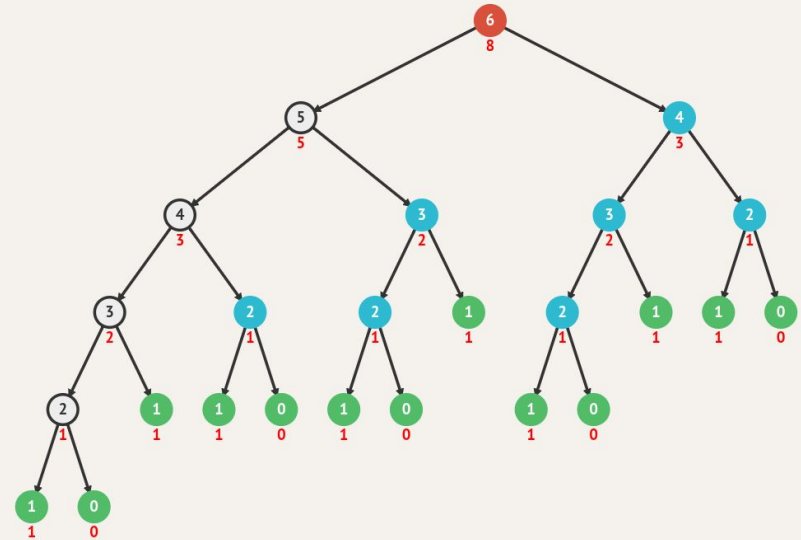
Fibonacci recursivo

Fibonacci Recursivo

```
1: function RECFIB( $n$ )  
2:   if  $n = 0$  then  
3:     return 0  
4:   else if  $n = 1$  then  
5:     return 1  
6:   else  
7:     return RECFIB( $n - 1$ ) + RECFIB( $n - 2$ )
```

$$T(n) = T(n-1) + T(n-2)$$

$$T(n) \in O(2^n)$$



fib(1) se calculó 8 veces

fib(2) se calculó 5 veces

fib(3) se calculó 3 veces

$n=20$ ~22.000 llamadas

$n=30$ ~2.7M de llamadas

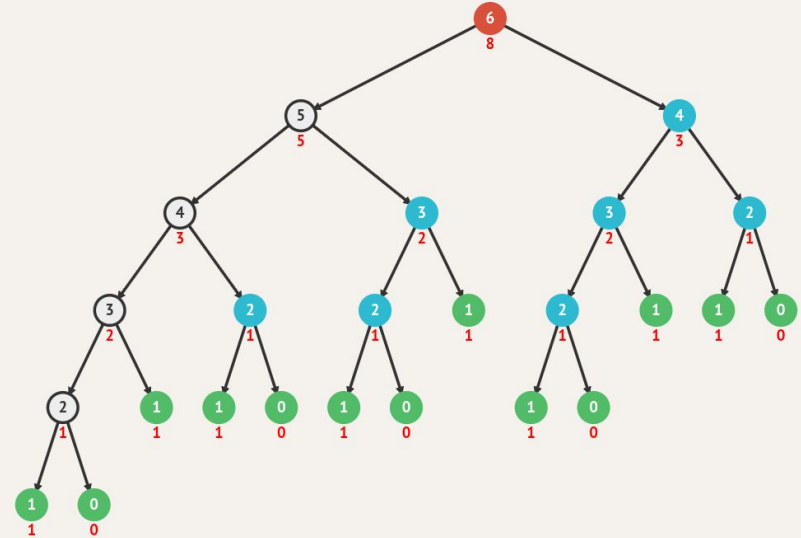
Fibonacci recursivo

Fibonacci Recursivo

```
1: function RECFIB( $n$ )  
2:   if  $n = 0$  then  
3:     return 0  
4:   else if  $n = 1$  then  
5:     return 1  
6:   else  
7:     return RECFIB( $n - 1$ ) + RECFIB( $n - 2$ )
```

$$T(n) = T(n-1) + T(n-2)$$

$$T(n) \in O(2^n)$$



fib(1) se calculó 8 veces $n=20$ ~22.000 llamadas

fib(2) se calculó 5 veces $n=30$ ~2.7M de llamadas

fib(3) se calculó 3 veces $n=50$ ~40.000M de llamadas

Contenido

- **Características y enfoques de la Programación dinámica**
 - Subestructura óptima y subproblemas superpuestos
 - Memoización y tabulación
- **Ejemplos**
 - Fibonacci
 - Problema de la mochila 0/1
 - Longest Common Subsequence (LCS)
 - Cambio de moneda

Características programación dinámica

Técnica del diseño de algoritmos que:

- **Resuelve** el problema dividiéndolo en **subproblemas superpuestos**
- Cada subproblema se resuelve una sola vez y su **resultado se almacena**
- **Evita recalcular** subproblemas ya resueltos almacenándolos en una tabla de memorización.

Para aplicar programación dinámica el algoritmo tiene que poder dividirse en subproblemas superpuestos y tener una subestructura óptima.

Características programación dinámica

Subproblemas superpuestos:

- Los mismos subproblemas se **resuelven múltiples veces** durante la ejecución del algoritmo recursivo.

Subestructura óptima:

- La solución óptima global puede construirse a partir de soluciones óptimas de sus subproblemas .
- Si no hay subestructura óptima, PD no garantiza una solución correcta.

Estructura

Estructura General de Programación Dinámica

```
1: function PD(problema)
2:   if solución ya calculada en tabla then
3:     return obtener_de_tabla(problema)
4:   if caso base then
5:     return resolver_directamente(problema)
6:   subproblemas  $\leftarrow$  dividir(problema)
7:   solución  $\leftarrow$   $\emptyset$ 
8:   for cada subproblema en subproblemas do
9:     solución  $\leftarrow$  combinar(solución, PD(subproblema))
10:  guardar_en_tabla(problema, solución)
11:  return solución
```

Enfoques de la programación dinámica

Top down (Memoización)

- Se resuelve el problema de forma recursiva
- Parte del problema original
- Usa recursión con caché
- Desventaja: Overhead de las llamadas recursivas

Bottom up (Tabulación)

- Normalmente se resuelve el problema de forma iterativa
- Se resuelven los subproblemas más pequeños primero
- Desventaja: A veces calcula subproblemas que no son necesarios

Enfoques de la programación dinámica

Top down (Memoización)

- Se resuelve el problema de forma recursiva
- Parte del problema original
- Usa recursión con caché
- Desventaja: Overhead de las llamadas recursivas

```
int memo[MAX];

int f(int n) {
    if (memo[n] != -1)
        return memo[n];

    if (caso_base)
        memo[n] = /* valor base */;
    else
        memo[n] = f(subproblema);

    return memo[n];
}
```

- Verificar si ya fue calculado
 - Si está en memo → retornar directamente
 - Si no → calcular y guardar en memo
- Retornar el valor almacenado

Enfoques de la programación dinámica

Bottom up (Tabulación)

- Normalmente se resuelve el problema de forma iterativa
- Se resuelven los subproblemas más pequeños primero
- Desventaja: A veces calcula subproblemas que no son necesarios

```
int f_tab(int n) {  
    int dp[n+1];  
  
    dp[0] = /* valor base 0 */;  
    dp[1] = /* valor base 1 */;  
  
    for (int i = 2; i <= n; i++)  
        dp[i] = /*max(dp[i-1], dp[i-2]); */  
                /*optimo(subproblemas);*/  
                /*dp[i-1] + dp[i-2] */  
  
    return dp[n];  
}
```

- Definir el tamaño de la tabla dp
- Inicializar casos base
- Llenar de forma ascendente (bottom-up)
- Usar valores ya calculados de la tabla

Programación dinámica

Ventajas

- **Evita** recálculos innecesarios
- Mejora la **eficiencia** cuando hay problemas superpuestos
- Garantiza la **solución óptima**

Desventajas

- Requiere **memoria** adicional
- **Identificar la estructura óptima** puede no ser tan intuitivo
- Mayor **complejidad al implementar** que soluciones ingenuas

Programación dinámica

Ventajas

- **Evita** recálculos innecesarios
- Mejora la **eficiencia** cuando hay problemas superpuestos
- Garantiza la **solución óptima**

Desventajas

- Requiere **memoria** adicional
- **Identificar la estructura óptima** puede no ser tan intuitivo
- Mayor **complejidad al implementar** que soluciones ingenuas

¿Cuándo utilizarla?

- Cuando el problema se puede dividir en problemas más pequeños
- Cuando hay problemas superpuestos
- Cuando hay una subestructura óptima
- Cuando se necesita optimizar una solución recursiva
- Cuando el problema busca un valor óptimo (máximo o mínimo)

Programación dinámica vs divide y vencerás

Aspecto	Divide y Vencerás	Programación Dinámica
Subproblemas	Independientes (no se repiten)	Superpuestos (se repiten)
Almacenamiento	No es necesario	Sí (memo array / tabla DP)
Estrategia	Dividir → Resolver → Combinar	Identificar subprob. → Almacenar → Construir
Dirección	Top-down siempre	Top-down (memo) o Bottom-up (tabla)
Complejidad típica	$O(n \log n)$	polinómica : $O(n)$, $O(n^2)$, ...
Ejemplos clásicos	MergeSort, QuickSort, BinSearch	Fibonacci, Mochila, LCS, Floyd-Warshall

Problema de la soga

Dada una soga de longitud n , se quiere encontrar los cortes que maximizan la ganancia total:

- podemos cortarla en piezas de longitud entera
- cada longitud i tiene un precio $p[i]$ conocido de antemano
- Podemos hacer cualquier número de cortes (incluyendo ninguno)

Longitud (i)	1	2	3	4	5	6	7	8
Precio (p[i])	1	5	8	9	10	17	17	20

Fibonacci recursivo

Fibonacci Recursivo

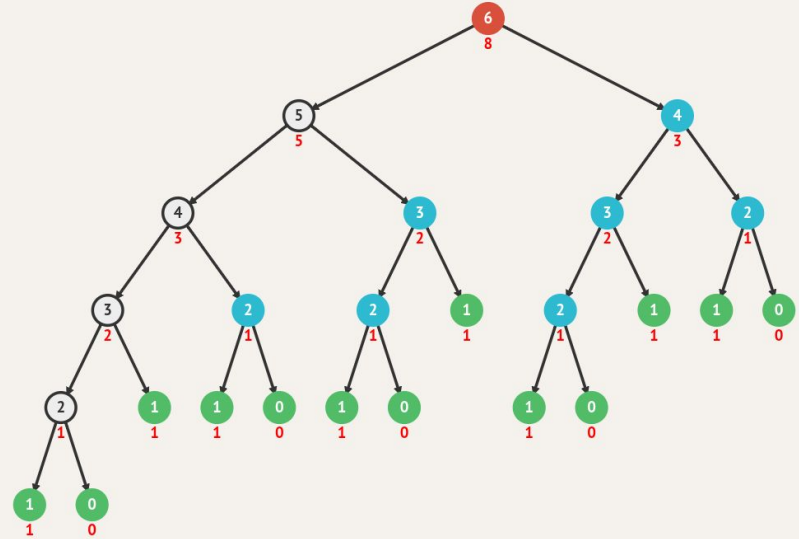
```

1: function RECFIB( $n$ )
2:   if  $n = 0$  then
3:     return 0
4:   else if  $n = 1$  then
5:     return 1
6:   else
7:     return RECFIB( $n - 1$ ) + RECFIB( $n - 2$ )

```

$$T(n) = T(n-1) + T(n-2)$$

$$T(n) \in O(2^n)$$



Fibonacci memoización

```
#define MAX 1000
int memo[MAX];

int fib_memo(int n) {
    if (memo[n] != -1)
        return memo[n];

    if (n <= 1)
        return memo[n] = n;

    return memo[n] =
        fib_memo(n-1) + fib_memo(n-2);
}
```

- Inicializar **memo** con -1
- Verificar si ya se calculó la solución:
 - si, retornar memo[n]
 - Si no, calcular y guardar
- Cada subproblema se calcula de manera recursiva **solo una vez**

Fibonacci memoización

```
#define MAX 1000
int memo[MAX];

int fib_memo(int n) {
    if (memo[n] != -1)
        return memo[n];

    if (n <= 1)
        return memo[n] = n;

    return memo[n] =
        fib_memo(n-1) + fib_memo(n-2);
}
```

- Inicializar **memo** con -1
- Verificar si ya se calculó la solución:
 - si, retornar memo[n]
 - Si no, calcular y guardar
- Cada subproblema se calcula de manera recursiva **solo una vez**

$$T(n) \in O(n)$$

Fibonacci tabulación

```
int fib_tab(int n) {  
    if (n <= 1) return n;  
  
    int dp[n+1];  
    dp[0] = 0; dp[1] = 1;  
  
    for (int i = 2; i <= n; i++)  
        dp[i] = dp[i-1] + dp[i-2];  
  
    return dp[n];  
}
```

- Llenar la tabla de **menor a mayor** de forma iterativa
- Cada celda se calcula **usando las dos anteriores**
- No hay recursión por lo que se recorre la tabla una sola vez
- Cada valor de la secuencia de fibonacci **se calcula una sola vez**

$$T(n) \in O(n)$$

Problema de la mochila (knapsack problem)

Tienes una mochila que soporta:

- un peso máximo de W kg y,
- un conjunto de n objetos (cada uno con un peso y un valor)

Problema a optimizar: ¿Qué objetos metes en la mochila para maximizar el valor total sin exceder el peso máximo? = **maximizar el valor total sin exceder W**

Observación: No puedes dividir los objetos, o llevas el objeto completo o lo dejas.

Objeto	Peso ($w[i]$)	Valor ($v[i]$)
Laptop	3kg	3
Cámara	2kg	4
Reloj	1kg	5
Tablet	4kg	7

Peso máximo permitido: 5kg

Problema de la mochila (knapsack problem)

- ¿Subproblemas?
- ¿Decisiones?
- ¿Casos bases?

Objeto	Peso ($w[i]$)	Valor ($v[i]$)
Laptop	3kg	3
Cámara	2kg	4
Reloj	1kg	5
Tablet	4kg	7

Maximizar el valor total sin exceder W

Peso máximo permitido: 5kg

Problema de la mochila (knapsack problem)

- Subproblema: $V(i, w)$ = máximo valor usando los primeros i objetos con capacidad w .

Objeto	Peso ($w[i]$)	Valor ($v[i]$)
Laptop	3kg	3
Cámara	2kg	4
Reloj	1kg	5
Tablet	4kg	7

Maximizar el valor total sin exceder W

Peso máximo permitido: 5kg

Problema de la mochila (knapsack problem)

- **Subproblema:** $V(i, w)$ = máximo valor usando los primeros i objetos con capacidad w .
- Para cada objeto i , tenemos **dos opciones**:
 - **No tomar objeto i :** $V(i, w) = V(i-1, w)$
 - **Tomar objeto i** (sólo si $w_i \leq w$): $V(i, w) = v_i + V(i-1, w-w_i)$
- **Tomamos el máximo** de ambas opciones
- La **solución final es $V(n, w)$**

Objeto	Peso ($w[i]$)	Valor ($v[i]$)
Laptop	3kg	3
Cámara	2kg	4
Reloj	1kg	5
Tablet	4kg	7

Maximizar el valor total sin exceder W

Peso máximo permitido: 5kg

Problema de la mochila (knapsack problem)

- **Subproblema:** $V(i, w)$ = máximo valor usando los primeros i objetos con capacidad w .
- Para cada objeto i , tenemos **dos opciones**:
 - **No tomar objeto i :** $V(i, w) = V(i-1, w)$
 - **Tomar objeto i** (sólo si $w_i \leq w$): $V(i, w) = v_i + V(i-1, w-w_i)$
- **Tomamos el máximo** de ambas opciones
- La **solución final es $V(n, w)$**
- Casos base:
 - $V(0, w)$ = sin objetos, valor = 0
 - $V(i, 0)$ = sin capacidad, valor = 0

Objeto	Peso ($w[i]$)	Valor ($v[i]$)
Laptop	3kg	3
Cámara	2kg	4
Reloj	1kg	5
Tablet	4kg	7

Maximizar el valor total sin exceder W
Peso máximo permitido: 5kg

Problema de la mochila (knapsack problem)

- **Subproblema:** $V(i, w)$ = máximo valor usando los primeros i objetos con capacidad w .
- Para cada objeto i , tenemos **dos opciones**:
 - **No tomar objeto i :** $V(i, w) = V(i-1, w)$
 - **Tomar objeto i** (sólo si $w_i \leq w$): $V(i, w) = v_i + V(i-1, w-w_i)$
- **Tomamos el máximo** de ambas opciones
- La **solución final es $V(n, w)$**
- Casos base:
 - $V(0, w)$ = sin objetos, valor = 0
 - $V(i, 0)$ = sin capacidad, valor = 0

Capacidad (w) – Peso máximo permitido: 5kg

Objeto (i)	0	1	2	3	4	5
$i = 0$ (sin objetos)						
 Laptop $i = 1$ (peso 3, valor 3)						
 Cámara $i = 2$ (peso 2, valor 4)						
 Reloj $i = 3$ (peso 1, valor 5)						
 Tablet $i = 4$ (peso 4, valor 7)						

Problema de la mochila (knapsack problem)

- **Subproblema:** $V(i, w)$ = máximo valor usando los primeros i objetos con capacidad w .
- Para cada objeto i , tenemos **dos opciones**:
 - **No tomar objeto i :** $V(i, w) = V(i-1, w)$
 - **Tomar objeto i** (sólo si $w_i \leq w$): $V(i, w) = v_i + V(i-1, w-w_i)$
- **Tomamos el máximo** de ambas opciones
- La **solución final es $V(n, w)$**
- Casos base:
 - $V(0, w)$ = sin objetos, valor = 0
 - $V(i, 0)$ = sin capacidad, valor = 0

Capacidad (w) – Peso máximo permitido: 5kg

Objeto (i)	0	1	2	3	4	5
$i = 0$ (sin objetos)	0	0	0	0	0	0
 Laptop $i = 1$ (peso 3, valor 3)	0	0	0	3	3	3
 Cámara $i = 2$ (peso 2, valor 4)	0	0	4	4	4	7
 Reloj $i = 3$ (peso 1, valor 5)	0	5	5	9	9	9
 Tablet $i = 4$ (peso 4, valor 7)	0	5	5	9	9	12

 Camino óptimo (una de las soluciones)

 Valor óptimo: 12

Problema de la mochila (knapsack problem)

¿Cómo reconstruimos la solución?

- Iniciamos en la **celda inferior derecha** $v(n, W)$
- **Determinamos si se tomó o no el objeto**
(comparamos la celda actual con la de arriba)
 - Si $v(i, w) = v(i-1, w) \rightarrow$ no tomamos el objeto i :
 - nos movemos hacia arriba $v(i-1, w)$
 - Si $v(i, w) \neq v(i-1, w) \rightarrow$ tomamos el objeto i :
 - nos movemos a $v(i-1, w-w_i)$
- Repetimos hasta llegar a $i = 0$ o $w = 0$
- **Los objetos tomados son los que conforman la solución óptima**

Capacidad (w) – Peso máximo permitido: 5kg

Objeto (i)	0	1	2	3	4	5
$i = 0$ (sin objetos)	0	0	0	0	0	0
 Laptop $i = 1$ (peso 3, valor 3)	0	0	0	3	3	3
 Cámara $i = 2$ (peso 2, valor 4)	0	0	4	4	4	7
 Reloj $i = 3$ (peso 1, valor 5)	0	5	5	9	9	9
 Tablet $i = 4$ (peso 4, valor 7)	0	5	5	9	9	12

 Camino óptimo (una de las soluciones)

 Valor óptimo: 12

Memoización: Problema de la mochila

```
int knapSackMemo (int W, int wt[], int val[], int n)
{
    if (n == 0 || W == 0)
        return 0;

    if (dp[n][W] != -1)
        return dp[n][W];

    if (wt[n - 1] > W)
        return dp[n][W] = knapSackMemo (W, wt, val, n - 1);

    else
        return dp[n][W]
            = max (val[n - 1] + knapSackMemo (W - wt[n - 1], wt,
                                                val, n - 1),
                  knapSackMemo (W, wt, val, n - 1));
}
```


Memoización: Problema de la mochila

```
int knapSackMemo (int W, int wt[], int val[], int n)
{
    if (n == 0 || W == 0)
        return 0;

    if (dp[n][W] != -1)
        return dp[n][W];

    if (wt[n - 1] > W)
        return dp[n][W] = knapSackMemo (W, wt, val, n - 1);

    else
        return dp[n][W]
            = max (val[n - 1] + knapSackMemo (W - wt[n - 1], wt,
                                                val, n - 1),
                  knapSackMemo (W, wt, val, n - 1));
}
```

Memoización: Problema de la mochila

```
int knapSackMemo (int W, int wt[], int val[], int n)
{
```

```
    if (n == 0 || W == 0)
        return 0;
```

casos base: no tengo objetos o no tengo capacidad

```
    if (dp[n][W] != -1)
        return dp[n][W];
```

```
    if (wt[n - 1] > W)
        return dp[n][W] = knapSackMemo (W, wt, val, n - 1);
```

```
    else
        return dp[n][W]
            = max (val[n - 1] + knapSackMemo (W - wt[n - 1], wt,
                                                val, n - 1),
                  knapSackMemo (W, wt, val, n - 1));
```

```
}
```

Memoización: Problema de la mochila

```
int knapSackMemo (int W, int wt[], int val[], int n)
{
```

```
    if (n == 0 || W == 0)
        return 0;
```

casos base: no tengo objetos o no tengo capacidad

```
    if (dp[n][W] != -1)
        return dp[n][W];
```

```
    if (wt[n - 1] > W)
        return dp[n][W] = knapSackMemo (W, wt, val, n - 1);

    else
        return dp[n][W]
            = max (val[n - 1] + knapSackMemo (W - wt[n - 1], wt,
                                                val, n - 1),
                  knapSackMemo (W, wt, val, n - 1));
}
```

Memoización: Problema de la mochila

```
int knapSackMemo (int W, int wt[], int val[], int n)
{
```

```
    if (n == 0 || W == 0)
        return 0;
```

casos base: no tengo objetos o no tengo capacidad

```
    if (dp[n][W] != -1)
        return dp[n][W];
```

ya se calculó la solución óptima

```
    if (wt[n - 1] > W)
        return dp[n][W] = knapSackMemo (W, wt, val, n - 1);

    else
        return dp[n][W]
            = max (val[n - 1] + knapSackMemo (W - wt[n - 1], wt,
                                                val, n - 1),
                  knapSackMemo (W, wt, val, n - 1));
```

```
}
```

Memoización: Problema de la mochila

```
int knapSackMemo (int W, int wt[], int val[], int n)
{
```

```
    if (n == 0 || W == 0)
        return 0;
```

casos base: no tengo objetos o no tengo capacidad

```
    if (dp[n][W] != -1)
        return dp[n][W];
```

ya se calculó la solución óptima

```
    if (wt[n - 1] > W)
        return dp[n][W] = knapSackMemo (W, wt, val, n - 1);
```

```
    else
```

```
        return dp[n][W]
            = max (val[n - 1] + knapSackMemo (W - wt[n - 1], wt,
                                                val, n - 1),
                  knapSackMemo (W, wt, val, n - 1));
```

```
}
```

Memoización: Problema de la mochila

```
int knapSackMemo (int W, int wt[], int val[], int n)
{
```

```
    if (n == 0 || W == 0)
        return 0;
```

casos base: no tengo objetos o no tengo capacidad

```
    if (dp[n][W] != -1)
        return dp[n][W];
```

ya se calculó la solución óptima

```
    if (wt[n - 1] > W)
        return dp[n][W] = knapSackMemo (W, wt, val, n - 1);
```

el objeto no cabe en la capacidad restante

```
    else
        return dp[n][W]
            = max (val[n - 1] + knapSackMemo (W - wt[n - 1], wt,
                                                val, n - 1),
                  knapSackMemo (W, wt, val, n - 1));
```

```
}
```

Memoización: Problema de la mochila

```
int knapSackMemo (int W, int wt[], int val[], int n)
```

```
{  
    if (n == 0 || W == 0)  
        return 0;
```

casos base: no tengo objetos o no tengo capacidad

```
    if (dp[n][W] != -1)  
        return dp[n][W];
```

ya se calculó la solución óptima

```
    if (wt[n - 1] > W)  
        return dp[n][W] = knapSackMemo (W, wt, val, n - 1);
```

el objeto no cabe en la capacidad restante

```
    else  
        return dp[n][W]  
            = max (val[n - 1] + knapSackMemo (W - wt[n - 1], wt,  
                                              val, n - 1),  
                  knapSackMemo (W, wt, val, n - 1));
```

```
}
```

Memoización: Problema de la mochila

```
int knapSackMemo (int W, int wt[], int val[], int n)
{
```

```
    if (n == 0 || W == 0)
        return 0;
```

casos base: no tengo objetos o no tengo capacidad

```
    if (dp[n][W] != -1)
        return dp[n][W];
```

ya se calculó la solución óptima

```
    if (wt[n - 1] > W)
        return dp[n][W] = knapSackMemo (W, wt, val, n - 1);
```

el objeto no cabe en la capacidad restante

```
    else
        return dp[n][W]
            = max (val[n - 1] + knapSackMemo (W - wt[n - 1], wt,
                                                val, n - 1),
                  knapSackMemo (W, wt, val, n - 1));
```

decisión: comparamos tomar o no
tomar el objeto y elegimos el máximo

```
}
```


Tabulación: Problema de la mochila

```
int knapSackTabulation(int W, int wt[], int val[], int n)
{
    int dp[n + 1][W + 1];
    for (int i = 0; i <= n; i++) {
        for (int w = 0; w <= W; w++) {
            if (i == 0 || w == 0)
                dp[i][w] = 0;
            else if (wt[i - 1] <= w)
                dp[i][w] = max(
                    val[i - 1] + dp[i - 1][w - wt[i - 1]],
                    dp[i - 1][w]);
            else
                dp[i][w] = dp[i - 1][w];
        }
    }

    return dp[n][W];
}
```

Tabulación: Problema de la mochila

```
int knapSackTabulation(int W, int wt[], int val[], int n)
{
    int dp[n + 1][W + 1];
    for (int i = 0; i <= n; i++) {
        for (int w = 0; w <= W; w++) {
            if (i == 0 || w == 0)
                dp[i][w] = 0;
            else if (wt[i - 1] <= w)
                dp[i][w] = max(
                    val[i - 1] + dp[i - 1][w - wt[i - 1]],
                    dp[i - 1][w]);
            else
                dp[i][w] = dp[i - 1][w];
        }
    }
    return dp[n][W];
}
```

casos base

decisión: tomar o no tomar el objeto

no cabe el objeto

Problema de la mochila (knapsack problem)

- Complejidad temporal:

Tenemos una tabla de tamaño $(n+1) \times (W+1)$

Para cada estado se comparan las dos opciones
($O(1)$)

$$T(n) \in O(nW)$$

El tiempo crece proporcionalmente al número de objetos por la capacidad.

Capacidad (w) – Peso máximo permitido: 5kg

Objeto (i)	0	1	2	3	4	5
$i = 0$ (sin objetos)	0	0	0	0	0	0
 Laptop $i = 1$ (peso 3, valor 3)	0	0	0	3	3	3
 Cámara $i = 2$ (peso 2, valor 4)	0	0	4	4	4	7
 Reloj $i = 3$ (peso 1, valor 5)	0	5	5	9	9	9
 Tablet $i = 4$ (peso 4, valor 7)	0	5	5	9	9	12

 Camino óptimo (una de las soluciones)

 Valor óptimo: 12

Cambio de moneda

Tienes un conjunto de monedas de diferentes valores:

- un conjunto de **n monedas** (cada una con un valor $v[i]$)
- un **monto objetivo M**

Problema a optimizar: ¿Cuál es el número mínimo de monedas necesarias para formar el monto M usando las monedas disponibles? = **minimizar la cantidad de monedas tal que la suma sea exactamente M**

Observación: Tienen una cantidad ilimitada de monedas

Objeto	Valor ($v[i]$)
Moneda 1	1
Moneda 2	2
Moneda 3	5

Mínimo número de monedas para obtener M

Monto objetivo: \$11

Cambio de moneda

- ¿Subproblemas?
- ¿Decisiones?
- ¿Casos bases?

Objeto	Valor ($v[i]$)
Moneda 1	1
Moneda 2	2
Moneda 3	5

Mínimo número de monedas para
obtener M

Monto objetivo: \$11

Cambio de moneda

- **Subproblema:** $V(i, M)$ = número mínimo de monedas para formar un monto M usando las primeras i monedas

Para cada moneda i , tenemos dos opciones:

- **No tomar la moneda i :** $V(i, M) = V(i-1, M)$
- **Tomar la moneda i** (sólo si $v_i \leq M$):
$$V(i, M) = 1 + V(i, M - v_i)$$
- **Tomamos el mínimo** de ambas opciones
- La **solución final es $V(n, M)$**
- Casos base:
 - $V(i, 0)$ = monto 0, no se requieren monedas
 - $V(0, M)$ = no hay monedas para formar un monto

Objeto	Valor ($v[i]$)
Moneda 1	1
Moneda 2	2
Moneda 3	5

Mínimo número de monedas para obtener M

Monto objetivo: \$11

Cambio de moneda

- **Subproblema:** $V(i, M)$ = número mínimo de **monedas** para formar un monto M usando las primeras i monedas

Para cada moneda i , tenemos dos opciones:

- **No tomar la moneda i :** $V(i, M) = V(i-1, M)$
- **Tomar la moneda i** (sólo si $v_i \leq M$):
$$V(i, M) = 1 + V(i, M - v_i)$$
- **Tomamos el mínimo** de ambas opciones
- La **solución final es $V(n, M)$**
- Casos base:
 - $V(i, 0)$ = monto 0, no se requieren monedas
 - $V(0, M)$ = no hay monedas para formar un monto



Monedas disponibles: {1, 2, 5}

Objetivo: **Monto = 11**

Monto (i)	Monedas consideradas (j)			
	0 (sin monedas)	1 (moneda = 1)	2 (moneda = 2)	3 (moneda = 5)
0				
1				
2				
3				
4				
5				
6				
7				
8				
9				
10				
11				

Cambio de moneda

- **Subproblema:** $V(i, M)$ = número mínimo de **monedas** para formar un monto M usando las primeras i monedas

Para cada moneda i , tenemos dos opciones:

- **No tomar la moneda i :** $V(i, M) = V(i-1, M)$
- **Tomar la moneda i** (sólo si $v_i \leq M$):
$$V(i, M) = 1 + V(i, M - v_i)$$
- **Tomamos el mínimo** de ambas opciones
- La **solución final es $V(n, M)$**
- Casos base:
 - $V(i, 0) =$ monto 0, no se requieren monedas
 - $V(0, M) =$ no hay monedas para formar un monto



Monedas disponibles: {1, 2, 5}

Objetivo: **Monto = 11**

Monto (i)	Monedas consideradas (j)			
	0 (sin monedas)	1 (moneda = 1)	2 (moneda = 2)	3 (moneda = 5)
0	0	0	0	0
1	∞	1	1	1
2	∞	2	1	1
3	∞	3	2	2
4	∞	4	2	2
5	∞	5	3	1
6	∞	6	3	2
7	∞	7	4	2
8	∞	8	4	3
9	∞	9	5	3
10	∞	10	5	2
11	∞	11	6	3

Cambio de moneda

¿Cómo reconstruimos la solución?

- Iniciamos en la **celda inferior derecha** $v(M,n)$
- **Determinamos que moneda se usó**
- (probamos cada moneda disponible)
 - Si $v(i,n) = 1 + v(i-coin_n, n) \rightarrow$ usamos la moneda:
 - nos movemos a $v(i-coin_n, n)$
(restamos la moneda)
 - Si no, nos movemos hacia la izquierda y probamos con la siguiente moneda (n-1) hasta encontrar alguna que cumpla.
- Repetimos hasta llegar a $i = 0$
- **Los monedas usadas forman la solución óptima**



Monedas disponibles: {1, 2, 5}

Objetivo: Monto = 11

Monto (i)	Monedas consideradas (j)			
	0 (sin monedas)	1 (moneda = 1)	2 (moneda = 2)	3 (moneda = 5)
0	0	0	0	0
1	∞	1	1	1
2	∞	2	1	1
3	∞	3	2	2
4	∞	4	2	2
5	∞	5	3	1
6	∞	6	3	2
7	∞	7	4	2
8	∞	8	4	3
9	∞	9	5	3
10	∞	10	5	2
11	∞	11	6	3

Memoización: Cambio de moneda

```
int memo[MAX];

int min(int a, int b) {
    return a < b ? a : b;
}

int coin_change_memo(int m) {
    if (m == 0) return 0;
    if (m < 0) return INT_MAX;

    if (memo[m] != -1)
        return memo[m];

    int mejor = INT_MAX;
    for (int i = 0; i < n_monedas; i++) {
        int sub = coin_change_memo(m - monedas[i]);
        if (sub != INT_MAX)
            mejor = min(mejor, sub + 1);
    }

    return memo[m] = mejor;
}
```



Monedas disponibles: {1, 2, 5}

Objetivo: Monto = 11

Monto (i)	Monedas consideradas (j)			
	0 (sin monedas)	1 (moneda = 1)	2 (moneda = 2)	3 (moneda = 5)
0	0	0	0	0
1	∞	1	1	1
2	∞	2	1	1
3	∞	3	2	2
4	∞	4	2	2
5	∞	5	3	1
6	∞	6	3	2
7	∞	7	4	2
8	∞	8	4	3
9	∞	9	5	3
10	∞	10	5	2
11	∞	11	6	3

Cambio de moneda

- **Subproblema:** $dp[M]$ = número mínimo de monedas para formar un monto M .
- **Decisión:** ¿Qué moneda usamos como última moneda?
 - Probamos todas las monedas disponibles como última opción y nos quedamos con la mejor (la que usa menos monedas)
- **Orden de evaluación de montos:** De 0 a M (cada valor de $dp[M]$ usa los valores ya calculados).
- Casos base:
 - $dp[0]$ = monto 0, no se requieren monedas



Monedas disponibles: {1, 2, 5}

Objetivo: **Monto = 11**

Monto (M)	0	1	2	3	4	5	6	7	8	9	10	11
dp[M]												

Cambio de moneda

- **Subproblema:** $dp[M]$ = número mínimo de monedas para formar un monto M .
- **Decisión:** ¿Qué moneda usamos como última moneda?
 - Probamos todas las monedas disponibles como última opción y nos quedamos con la mejor (la que usa menos monedas)
- **Orden de evaluación de montos:** De 0 a M (cada valor de $dp[M]$ usa los valores ya calculados).
- Casos base:
 - $dp[0]$ = monto 0, no se requieren monedas



Monedas disponibles: {1, 2, 5}

Objetivo: **Monto = 11**

Monto (M)	0	1	2	3	4	5	6	7	8	9	10	11
dp[M]	0	1	1	2	2	1	2	2	3	3	2	3

M	Decisiones (probar cada moneda como última)	Cálculos	Mínimo	Resultado
1	• Usar 1: $1 + dp[1 - 1] = 1 + dp[0]$	$1 + 0 = 1$	1	$dp[1] = 1$ (usar 1)
2	• Usar 1: $1 + dp[2 - 1] = 1 + dp[1]$ • Usar 2: $1 + dp[2 - 2] = 1 + dp[0]$	$1 + 1 = 2$ $1 + 0 = 1$	1	$dp[2] = 1$ (usar 2)
3	• Usar 1: $1 + dp[3 - 1] = 1 + dp[2]$ • Usar 2: $1 + dp[3 - 2] = 1 + dp[1]$	$1 + 1 = 2$ $1 + 1 = 2$	2	$dp[3] = 2$ (usar 1 o 2)
4	• Usar 1: $1 + dp[4 - 1] = 1 + dp[3]$ • Usar 2: $1 + dp[4 - 2] = 1 + dp[2]$	$1 + 2 = 3$ $1 + 1 = 2$	2	$dp[4] = 2$ (usar 2)
5	• Usar 1: $1 + dp[5 - 1] = 1 + dp[4]$ • Usar 2: $1 + dp[5 - 2] = 1 + dp[3]$ • Usar 5: $1 + dp[5 - 5] = 1 + dp[0]$	$1 + 2 = 3$ $1 + 2 = 3$ $1 + 0 = 1$	1	$dp[5] = 1$ (usar 5)
...	...	...	...	...
10	• Usar 1: $1 + dp[10 - 1] = 1 + dp[9]$ • Usar 2: $1 + dp[10 - 2] = 1 + dp[8]$ • Usar 5: $1 + dp[10 - 5] = 1 + dp[5]$	$1 + 3 = 4$ $1 + 3 = 4$ $1 + 1 = 2$	2	$dp[10] = 2$ (usar 5)
11	• Usar 1: $1 + dp[11 - 1] = 1 + dp[10]$ • Usar 2: $1 + dp[11 - 2] = 1 + dp[9]$ • Usar 5: $1 + dp[11 - 5] = 1 + dp[6]$	$1 + 2 = 3$ $1 + 3 = 4$ $1 + 2 = 3$	3	$dp[11] = 3$ (usar 1 o 5)

Cambio de moneda

¿Cómo reconstruimos la solución?

- Iniciamos en el monto objetivo ($M = 11$)
- **Probamos cada moneda disponible**
 - Si $dp[M] = 1 + dp[M-c] \rightarrow c$ pudo ser la última moneda utilizada
 - Usamos esa moneda y nos movemos a $M-c$
- Probamos las monedas hasta encontrar una que reproduzca el valor óptimo
- Repetimos hasta llegar a $M = 0$
- **Los monedas usadas forman la solución óptima**

Monedas disponibles: {1, 2, 5}

Objetivo: **Monto = 11**

Monto (M)	0	1	2	3	4	5	6	7	8	9	10	11
dp[M]	0	1	1	2	2	1	2	2	3	3	2	3

Paso	Monto actual (M)	Probamos moneda (c)	¿Cumple $dp[M] = 1 + dp[M - c]$?	Acción	Moneda elegida	
1	11	1	$dp[11] = 3$ y $1 + dp[10] = 1 + 2 = 3$ ✓	Usamos 5 (podríamos usar 1 también)	5	
		2	$dp[11] = 3$ y $1 + dp[9] = 1 + 3 = 4$ ✗			
		5	$dp[11] = 3$ y $1 + dp[6] = 1 + 2 = 3$ ✓			
2	6	1	$dp[6] = 2$ y $1 + dp[5] = 1 + 1 = 2$ ✓	Usamos 5	5	
		2	$dp[6] = 2$ y $1 + dp[4] = 1 + 2 = 3$ ✗			
		5	$dp[6] = 2$ y $1 + dp[1] = 1 + 1 = 2$ ✓			
3	1	1	$dp[1] = 1$ y $1 + dp[0] = 1 + 0 = 1$ ✓	Usamos 1	1	
		2	$dp[1] = 1$ y $1 + dp[-1]$ ✗ (no válido)			
		5	$dp[1] = 1$ y $1 + dp[-4]$ ✗ (no válido)			
4	0	¡Hemos llegado a 0!				Fin

Memoización: Cambio de moneda

```
int memo[MAX];

int min(int a, int b) {
    return a < b ? a : b;
}

int coin_change_memo (int m) {
    if (m == 0) return 0;
    if (m < 0) return INT_MAX;

    if (memo[m] != -1)
        return memo[m];

    int mejor = INT_MAX;
    for (int i = 0; i < n_monedas; i++) {
        int sub = coin_change_memo (m - monedas[i]);
        if (sub != INT_MAX)
            mejor = min(mejor, sub + 1);
    }

    return memo[m] = mejor;
}
```

Memoización: Cambio de moneda

```
int memo[MAX];

int min(int a, int b) {
    return a < b ? a : b;
}

int coin_change_memo(int m) {
    if (m == 0) return 0;
    if (m < 0) return INT_MAX;

    if (memo[m] != -1)
        return memo[m];

    int mejor = INT_MAX;
    for (int i = 0; i < n_monedas; i++) {
        int sub = coin_change_memo(m - monedas[i]);
        if (sub != INT_MAX)
            mejor = min(mejor, sub + 1);
    }

    return memo[m] = mejor;
}
```

casos base: no tengo objetos o no tengo capacidad

Memoización: Cambio de moneda

```
int memo[MAX];

int min(int a, int b) {
    return a < b ? a : b;
}

int coin_change_memo(int m) {
    if (m == 0) return 0;
    if (m < 0) return INT_MAX;

    if (memo[m] != -1)
        return memo[m];

    int mejor = INT_MAX;
    for (int i = 0; i < n_monedas; i++) {
        int sub = coin_change_memo(m - monedas[i]);
        if (sub != INT_MAX)
            mejor = min(mejor, sub + 1);
    }

    return memo[m] = mejor;
}
```

casos base: no tengo objetos o no tengo capacidad

ya se calculó la solución óptima

Memoización: Cambio de moneda

```
int memo[MAX];

int min(int a, int b) {
    return a < b ? a : b;
}

int coin_change_memo (int m) {
    if (m == 0) return 0;
    if (m < 0) return INT_MAX;

    if (memo[m] != -1)
        return memo[m];

    int mejor = INT_MAX;
    for (int i = 0; i < n_monedas; i++) {
        int sub = coin_change_memo (m - monedas[i]);
        if (sub != INT_MAX)
            mejor = min(mejor, sub + 1);
    }

    return memo[m] = mejor;
}
```

casos base: no tengo objetos o no tengo capacidad

ya se calculó la solución óptima

decisión: tomamos la moneda?

Tabulación: Cambio de moneda

```
int monedas[] = {1, 2, 5, 10};
int n_monedas = 4;
int coin_change_tab(int monto) {
    int dp[monto + 1];

    dp[0] = 0;
    for (int i = 1; i <= monto; i++)
        dp[i] = INT_MAX;

    for (int m = 1; m <= monto; m++) {
        for (int i = 0; i < n_monedas; i++) {
            if (monedas[i] <= m && dp[m - monedas[i]] != INT_MAX) {
                int candidato = dp[m - monedas[i]] + 1;
                if (candidato < dp[m])
                    dp[m] = candidato;
            }
        }
    }

    return dp[monto] == INT_MAX ? -1 : dp[monto];
}
```

Tabulación: Cambio de moneda

```
int monedas[] = {1, 2, 5, 10};  
int n_monedas = 4;  
int coin_change_tab(int monto) {  
    int dp[monto + 1];
```

```
    dp[0] = 0;  
    for (int i = 1; i <= monto; i++)  
        dp[i] = INT_MAX;
```

casos base e inicialización

```
    for (int m = 1; m <= monto; m++) {  
        for (int i = 0; i < n_monedas; i++) {
```

```
            if (monedas[i] <= m && dp[m - monedas[i]] != INT_MAX) {  
                int candidato = dp[m - monedas[i]] + 1;  
                if (candidato < dp[m])  
                    dp[m] = candidato;  
            }
```

decisión: probamos cada moneda,
para ver cual tomar

```
        }  
    }  
    return dp[monto] == INT_MAX ? -1 : dp[monto];  
}
```

Cambio de moneda

- **Complejidad temporal:**

Tenemos una tabla de tamaño $(M+1)$.

Por cada monto probamos las monedas disponibles.

$$T(n) \in O(nM)$$

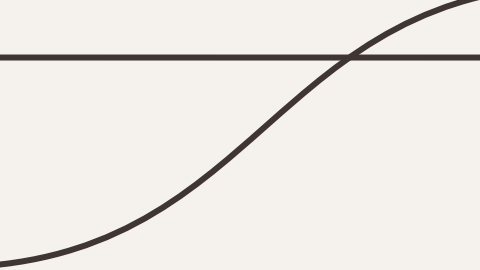
El tiempo crece proporcionalmente al monto objetivo y al número de monedas disponibles.



Monedas disponibles: {1, 2, 5}

Objetivo: **Monto = 11**

Monto (M)	0	1	2	3	4	5	6	7	8	9	10	11
dp[M]	0	1	1	2	2	1	2	2	3	3	2	3



Programación dinámica

Diseño de algoritmos
Jacqueline Aldridge Águila

